Talent Q | Vision Addendum 08

Scheduling, correspondence, J7 career intelligence, durable Q memory, orchestration and free design tooling

Status: binding addition to the founder's product vision and design scope, dated 27 October 2026. This document **extends** the earlier master and atlases; it neither deletes features nor claims these services are already integrated. The `00_CLAUDE_START_HERE_v2.md` directive supersedes `05_CLAUDE_DESIGN_FIRST_MASTER_DIRECTIVE.md` where they differ.

1. Decisions and source hierarchy

1. Brand is **Talent Q**, consistently spaced. Talent Q stands independently in public-facing material; shared Q reuse is an **internal implementation decision**.
2. Q must support recruiter-candidate correspondence and full scheduling, not merely show calendar links.
3. **J7 means candidate career development** in the master research. It is a **first-class Career Intelligence and Development product surface** requiring its own deep specification, screens, journeys, metrics and design reviews, not one item under onboarding.
4. Context and memory are a central design and engineering priority. Q should not repeatedly ask for correctly stored, permitted, current facts, yet must be able to correct, expire, forget, and challenge them.
5. Existing Q on `neo1415/q` already references LangGraph and has a run ledger, context firewall, versioned prompts, tools and approvals. **Audit and reuse those implementations** rather than adding a second LangGraph stack or training a new Q from scratch.
6. This is a **design-first deliverable**, not an instruction to deploy production services or spend on provider APIs. Prototype with accurate event contracts and clearly labeled mock data.
7. Figma is **optional**. Free Starter accounts have restricted access to write-to-canvas via MCP. Canonical deliverables must be **editable source code, component library, clickable prototype, visual regression captures, storyboards and recorded walkthroughs** in Git. Claude Design/Artifacts may be used as previews, not as the only copy of source files.

2. First-class subsystem A: Recruitment Scheduling and Correspondence (RSC)

Product promise

A recruiter asks, "Q, schedule interviews with these four candidates next week; avoid my Tuesday review, offer mornings first, confirm the panel, and send each candidate the correct briefing." Q handles **availability, preferences, proposals, correspondence, confirmations, reminders, changes, cancellations and pipeline state**, without exposing private calendar event details.

Actors and permissions

Recruiter, candidate, hiring manager, recruiting coordinator, interviewer/panel, organization administrator, Q-as-delegated-assistant, integration/provider, and agency operating on behalf of customer. Each principal has explicit organization, campaign, role and correspondence permissions. A candidate may choose time via link, email reply, text chat, voice conversation or other supported channel.

End-to-end state machine

```
DRAFT_REQUEST -> POLICY_AND_CONSENT_CHECK -> CALENDAR_CONNECTION_CHECK -> AVAILABILITY_READ -> SLOTS_CALCULATED -> RECRUITER_CONFIRMATION (if required) -> CANDIDATE_PROPOSAL_SENT -> AWAITING_CANDIDATE -> CANDIDATE_ACCEPTS / COUNTERPROPOSES / DECLINES / NO_RESPONSE -> AVAILABILITY_RECHECK -> EVENT_WRITE -> PROVIDER_ACK -> BOOKED -> REMINDERS -> COMPLETED | RESCHEDULED | CANCELLED | FAILED
```

A candidate choosing a time is **not** a booking until provider event creation succeeds. Q must not tell either party a meeting is booked on the strength of a draft or an email alone.

Capability requirements

- Secure OAuth integrations with **Google Calendar** and **Microsoft Outlook Calendar** where available; calendar account and permission health, organization admin policies, token renewal, revoked consent and downgrade handling.
- Availability-only scopes where possible; when broader scopes are needed for event creation, request them separately and clearly. Never include private event names, attendees, descriptions or conflicts in candidate-facing messages.
- Find group availability across recruiter and interview panel; configurable working hours, duration, timezone and DST, buffers, lead time, no-meeting blocks, holidays, interviewer load, regional accessibility and candidate location.

- Search and rank a bounded set of suitable time options, not an arbitrary endless list. Candidate can counterpropose. Recruiter can impose protected holds or require approval.
- Recheck free/busy **immediately before** committing calendar events. Use the provider's event version/ETag where applicable and idempotency keys. Compensate when one provider write succeeds but a second write fails. Avoid duplicate bookings after retries.
- Conference links (Google Meet, Teams or permitted provider), video/phone/onsite alternatives, candidate preparation instructions, accommodations and reschedule self-service.
- Correspondence across email, in-app, optional SMS, WhatsApp Business messaging and eligible voice channels. Each action respects user channel preference, country eligibility, opt-in, rate limit, template and budget.
- Versioned organization tone, approved templates, recruiter guidelines, legal notices, signatures, do-not-contact rules, candidate preferences and mandatory disclosures; **policy resolved before drafting, and rechecked before sending**.
- Delegation modes: `DRAFT_ONLY`, `APPROVE_EACH_SEND`, `APPROVED_CAMPAIGN` with audience, template, budget and expiry limits. Auto-send only under explicitly bounded authority; consequential changes have logs.
- Inbox threading across channels, contact deduplication, reply intent classification (accept, counteroffer, needs accommodation, withdraw, unsubscribe), human escalation, follow-up cadence, bounce/undeliverable tracking, opt-out honoring and no-response expiry.
- Consent and privacy: safe handling of candidate data in calendar descriptions and outbound text; only attach documents already approved for that recipient; make timezone explicit to all parties.
- Pipeline integration: `INTERVIEW_PROPOSED`, `INVITED`, `BOOKED`, `RESCHEDULE_REQUESTED`, `COMPLETED` events, distinct from message delivery statuses. Audit who approved what.

Example scheduling algorithm (concept model, not validated research)

1. `AvailableSlots = WorkingWindows ∩ RecruiterFree ∩ PanelFree ∩ CandidateAllowedWindows - Buffers - Constraints`.
2. Score feasible slot `s` with an interpretable objective such as `score(s)=w1*availability_preference+w2*timezone_fairness+w3*lead_time+w4*panel_load_balance`; each term normalized and explicitly defined per mandate.
3. Surface 3 to 5 options; do not expose restricted calendar metadata. Redis-backed short TTL provisional slot caching is optional, **never a durable booking source of truth**.

4. On candidate acceptance: revalidate provider free/busy -> commit event(s) idempotently -> persist meeting status -> send confirmations only after successful creation -> schedule reminders.

Hard scenarios to prototype

A) recruiter connected Google but panel member uses Outlook; B) candidate in another time zone and DST changes; C) slot is taken after proposal; D) recruiter disconnects account; E) two candidates accept same slot simultaneously; F) partial provider outage; G) candidate requests accessible text rather than video; H) WhatsApp channel cannot be used; I) interviewer absent and Q proposes replacement; J) candidate declines and requests future follow-up; K) organizer cancels; L) job closes before interview; M) message opt-out; N) review/approval delays; O) provider event succeeds but acknowledgement times out.

New design surfaces

Calendar connection and scope education; recruiter availability preferences; panel assignment; scheduler inside Q conversation; recruiter slot-preview/approval; candidate slot picker; availability warning; confirmed interview; cross-timezone comparison; correspondence inbox and templates; agent follow-up plan; reminders; reschedule; cancellation; conflict resolution; channel fallback and opt-out; audit and action receipts; offline/error/retry.

Research and integration sources

- Google Calendar FreeBusy: https://developers.google.com/workspace/calendar/api/v3/reference/freebusy/query
- Google Calendar event creation: https://developers.google.com/workspace/calendar/api/v3/reference/events/insert
- Microsoft Graph getSchedule: https://learn.microsoft.com/en-us/graph/outlook-get-free-busy-schedule
- Microsoft Graph findMeetingTimes, delegated work/school limitation: https://learn.microsoft.com/en-us/graph/api/user-findmeetingtimes?view=graph-rest-1.0
- WhatsApp Business Calling restrictions: https://www.twilio.com/docs/voice/whatsapp-business-calling

Important: WhatsApp **outbound business calling** currently excludes senders based in countries including Nigeria and the United States under cited provider guidance. Do not promise universally available WhatsApp calls. Offer alternatives and verify the sender/provider's current policy before planning integration.

3. First-class subsystem B: J7 Career Intelligence and Development

Identity and positioning

J7 is **not just an LMS** and is not simply mock interviews. It is a candidate-controlled **Career Intelligence Workspace**: helps a person understand work they want, skills they can demonstrate, opportunities they are realistic for, capability gaps that matter, the fastest credible ways to close those gaps, and how to show improvement to employers **with permission**.

Six connected engines, one experience

B1 Career Navigator. Role-family taxonomy; career goals, salary/location/remote/work-style preferences; paths from current competency evidence to target roles; multiple possible adjacent jobs; reality checks without demeaning the candidate. No sensitive-trait assumptions.

B2 Gap Explainer. Distinguish `NOT_EVIDENCED`, `BELOW_TARGET_ON_ASSESSMENT`, `EVIDENCE_OUTDATED`, `REQUIREMENT_UNCLEAR`, `VERIFIED_CAPABILITY`. Do not collapse lack of evidence into lack of ability. Explain each gap against a specific target-role requirement.

B3 Personal Growth Planner. One-week/two-week/monthly goal packages; realistic time and budget constraints; external curated resources, project prompts, mentoring and practical exercises. Learning plan is editable and skippable; user owns timing.

B4 Practice and Coaching Studio. Interview rehearsal in voice/text/forms with flexible follow-ups, real-work simulations by profession, question-specific feedback, structured behavioral evidence, improvement suggestions, examples and reattempts. Separate practice scores from employment evaluation scores and historical recruiter impressions.

B5 Evidence Builder. Turn consented projects, assignments, authored work, credentials and skills assessments into a corrected, provenance-tagged professional evidence passport. The user explicitly approves when a piece becomes recruiter-visible. No automatic publication of practice mistakes.

B6 Progress and Opportunity Loop. Explain evidence growth, assessment improvements under comparable conditions, time spent, target-readiness components, changing job recommendations and next useful actions. User can stop, export, delete and change goals.

Learning-science foundations

Use **retrieval practice** and **spaced practice** where relevant, not arbitrary gamified streaks. Give actionable, criterion-referenced feedback; let a candidate perform new work and demonstrate learning. See Carpenter, Pan & Butler, *Nature Reviews Psychology*

(2022): https://www.nature.com/articles/s44159-022-00089-1 . Research does **not** justify universal percentile rankings of personal worth or automatically inferring fitness from voice emotion.

J7 journeys to design with appropriate depth

1. First-time graduate: CV has few credentials -> Q identifies transferable project evidence -> creates credible beginner target map -> helps build practice project -> supports job search and interview prep.
2. Career switcher: retail to sales development, nurse to health-tech operations, or general IT to SOC. Different starting points and realistic learning paths.
3. Technical specialist: engineer uploads GitHub project; safe inspection; Q translates strengths to plain language; requests explicit permission to share; identifies limits of static code review.
4. Candidate rejected after interview: private constructive coaching from permitted feedback (not secret panel notes), plan, re-practice and new applications.
5. Experienced professional: target senior role, evidence of leadership and work outcomes, gaps in proof, selective sharing and anonymized exploration.
6. Low bandwidth: text-first coaching, downloadable plan, resume across devices, no avatar required.
7. Candidate changes career goal: history retained where permitted, previous path archived, search/preferences updated explicitly, unrelated recruiters cannot infer private interests.
8. Employer-sponsored growth: organization may fund learning; consent and private practice boundaries remain separate from employer monitoring.

J7 scoring architecture: distinct, job-related measurements

Do **not** create one global Talent Score. Design separate named measures:

- `evidence_coverage(role, candidate)` for requirement evidence completeness;
- `competency_mastery(assessment, rubric_version)` where validated, with reported conditions and uncertainty;
- `practice_progress(candidate, skill)` measured on comparable tasks and time intervals;
- `role_fit(candidate, mandate)` for a specific position and known preferences;
- `readiness_to_apply(target_role)` as an *advisory checklist*, not hiring eligibility;
- `growth_plan_adherence` as an optional private productivity metric, not a hiring filter.

Candidate-facing surfaces should distinguish **skill**, **evidence of skill**, **confidence**, **employment preference**, **progress** and **privacy status**. Fairness evaluation across roles, regions and accommodations is required before any score affects hiring.

Dedicated J7 screens and content architecture

Career home; goals/preferences; role explorer; transition paths; role comparison; readiness and evidence coverage; gap explainer; growth plan; study/practice resource library; practice interview; practical exercise; coaching transcript and annotation; private feedback history; assessment evidence ledger; before/after comparison; portfolio improvement wizard; evidence-share approval; opportunity re-discovery; mentorship/ partner connector concept; progress preferences; reminders; accessibility/low-data; assessment dispute and correction.

J7 standalone design review gate

Produce a **Career Intelligence standalone chapter** with its own journey maps, at least 3 complete prototypes (graduate, switcher, practiced specialist), mobile/text alternatives, failure states, sample evidence records, learning/assessment privacy model and acceptance tests. A generic dashboard and one pretty career progress chart do **not** count.

4. First-class subsystem C: Q Context and Memory

The promise, with correct limits

Q should remember *authorized, relevant and valid* previous information and show when it does so. It should be possible to ask: "Remember the availability rule I set for this team," "Which interview question did we agree on?" or "Why did you recommend this candidate last Tuesday?" Q responds using source-bound retrieval, version history and current permission checks. **It cannot promise literal perfect recall of all conversations indefinitely.**

Seven distinct layers (do not conflate)

1. **Live interaction state**: current turn, run IDs, tool events, selected entity, open UI artifact and temporary plan. Stored in Q run/checkpoint system, not treated as durable preferences.
2. **Conversation history**: user-visible message log with searchable references, source timestamps, edits and retention.
3. **Session summaries / context packs**: compact, cited snapshots of relevant past work; avoid rereading full chat each time.
4. **Durable preferences and explicit instructions**: candidate job preferences, recruiter meeting hours, organization standards and communication style. Versioned, scoped, inspectable, correctable.

5. **Entity knowledge and evidence graph**: role requirements, projects, companies, candidates, evidence, provenance, claims, counterclaims, temporal validity and sharing permissions. Not all information becomes 'memory' just because Q read it.
6. **Episodic task memory**: what Q tried, decisions, approvals, last processed cursor, provider receipts, success/failure, next steps. Key for long-running campaigns and scheduled work.
7. **Evaluated learned behavior**: anonymized/consented aggregated telemetry, human feedback, prompts/policies and model/ranking updates passing validation. **Not** arbitrary self-editing of prompts, model weights or permissions during a user chat.

Memory data contract (prototype and architecture specification)

`memory_id`, `subject_type/id`, `tenant_id`, `visibility_scope`, `owner_id`, `source_refs`, `source_timestamp`, `recorded_at`, `last_confirmed_at`, `expires_at/retention_class`, `truth_class`, `evidence_status`, `lifecycle_status`, `importance_reason`, `confidence_basis`, `consent_and_purpose`, `supersedes_id`, `payload_schema_version`, `retrieval_tags`, `encryption_class`, `delete_or_revoke_state`.

Source hierarchy, authorization and time take priority over embedding similarity. A stale remembered preference must not override a fresh user instruction or explicit organization policy. Private candidate practice must never become employer-side scoring or ranking input by accident.

Memory write algorithm (proposal)

1. Identify a candidate fact/preference/rule/task decision **without direct persistence by the model**.
2. Classify: explicit user save; task-continuity requirement; material preference; ephemeral statement; third-party claim; uncertain inference.
3. Validate against permitted sources and active purpose/consent. Discard disallowed, redundant and unsupported claims; request confirmation for consequential uncertain facts.
4. Assign owner and visibility scope, retention policy, expiry, source, version and correction route.
5. Deterministic **write gate** persists append/revision record. Emit a memory event, update derived indexes and invalidate affected caches.
6. Periodic maintenance expires stale items, reconciles contradictions and prompts for re-confirmation where valuable. No unbounded memory accumulation.

Memory retrieval algorithm (proposal)

`Access check -> task/entity routing -> structured facts + scoped knowledge graph -> lexical/semantic lookup -> time/source validity -> conflict detection -> relevance/diversity scoring -> context budget packing -> answer with provenance`.

Protect against cross-tenant leakage, sensitive-inference recombination, prompt injection from uploaded guidelines, stale snapshots, overly broad retrieval and serving data after revocation. Summarization and caching must carry **scope and revision identifiers**, not just text similarity.

Retention categories (design hypotheses, not default legal policy)

- Run-level: life of job plus short diagnostic retention under privacy policy.
- Temporary task context: bounded TTL, renewable by authorized task ownership.
- Confirmed preferences: until changed, revoked or policy retention applies; periodic re-confirmation.
- Org guidelines: versioned until superseded/removed.
- Interview/evaluation records: org/country-specific defined retention, access controls and deletion rules.
- Aggregated product learnings: only with applicable lawful basis and reviewed de-identification; never simply hoard recordings/transcripts.

Do **not** hardcode arbitrary 30/90/365-day retention as universal law. Produce a policy matrix for jurisdictions later.

Obsidian, Graphify, LangChain and LangGraph: different jobs

- **Obsidian graph view** visualizes links among notes in a human-managed vault. It is a useful mental model and optional research/architecture workspace, not Talent Q's secure production memory database. https://obsidian.md/help/plugins/graph
- **Graphify** turns a source codebase and documents into a queryable knowledge graph for **developers and coding agents**. Use it *experimentally* during the Q reuse audit to trace dependencies. Do not make Graphify mandatory on every user query or pretend it is a candidate memory database. https://github.com/Graphify-Labs/graphify
- **LangChain** provides components, model/tool abstractions and retrieval; use where Q already depends on it and adapters justify it. Do not introduce another generic framework layer without need.
- **LangGraph** provides checkpointing and durable graph execution; Q already imports `@langchain/langgraph` in `packages/q-orchestrator`. Reuse its installed graph/checkpoint mechanisms after inspecting actual implementation and version. Its

checkpointers handle thread state while stores can support cross-thread memory, but canonical verified facts still belong in governed domain storage. https://docs.langchain.com/oss/javascript/langgraph/persistence

Memory test/UX coverage

The user can see why Q remembers a detail, edit/correct it, change the audience, revoke it and verify future chats comply. Tests cover wrong-company context, wrong-recruiter calendar, shared vs private guidelines, after-revocation cached reuse, outdated salary preference, contradictory CV entries, resumed interview, candidate changing job target, two concurrent Q agents, expired task, interrupted background research, hallucinated recall and memory export/deletion.

5. Free-Figma design production that still produces real handoff assets

Preferred: local **code-first design system and interactive prototype** in a clean version-controlled React/Next.js workspace with semantic tokens, Storybook or a lightweight component gallery, route fixtures and responsive pages. Motion for React for ordinary animation; Rive only when it solves a concrete state-driven need. Playwright records real prototype user journeys to MP4 via available capture tools; provide actual source animation states and reduced-motion alternatives.

Claude Design/Artifacts: suitable for first visual directions, clickable illustrative designs and stakeholder review, but **not** the sole source of code, screen coverage or integration contracts. Export or reproduce accepted designs as editable repo code. Claude's official docs say Design template/Artifacts available across free and paid plans. https://support.claude.com/en/articles/17153992-what-are-artifacts-and-how-do-i-use-them

Free Figma Starter: usable for manual sketches/drafts, but automated MCP reads are limited (officially up to 20/month as of this research) and Figma's write-to-canvas requires eligible paid/education plans. The design build must not get blocked on this. https://help.figma.com/hc/en-us/articles/39252411778583-Figma-MCP-server-FAQs ; https://help.figma.com/hc/en-us/articles/39216419318551-Get-started-with-the-Figma-MCP-server

Output sequence:

1. Design direction, content style, reusable tokens and Q character/visual behavior.
2. Working code-based component gallery and 10-12 priority screen states.
3. Eight original golden journeys **plus** deep J7 and complete scheduling, correspondence and memory flows.
4. Screens, interaction states, keyboard/mobile/reduced-motion behavior, source traceability, tests and recordings.

5. Optional Figma import/replication later. Avoid recreating all prototype pages manually in Figma for the sake of a tool checkbox.

6. Architectural and cost guardrails

- One Q, bounded specialist capabilities. Parallel **read-only** tasks where useful; write effects remain permission-bound and idempotent.
- Q's agent planner must not self-grant new tools, exceed spend caps, bypass contracts or access arbitrary network addresses/files. Safe browsing uses allowlisted egress, sandboxed downloads and malware/content-type scans. Untrusted web content cannot become policy.
- Per-user/tenant/provider quotas and model budgets. Prompt caching for stable policy prefixes, retrieval/result caching with permission keys and revision invalidation, database projections and event-driven UI. No AI call for pagination, elementary filtering, status transitions or timezone arithmetic.
- Meaningful progress states derived from recorded job events. Avoid fake 'thinking' displays and hidden multi-minute busy indicators.
- Ship customer value early: a hiring-intake -> shortlist -> consented invitation -> booked interview -> review slice; a candidate import -> evidence -> practice -> growth plan slice. Build detailed **design** for full future scope without fully implementing it at MVP.
- Privacy, consent, region constraints, fairness, accessible alternatives and human review are release criteria for consequential recruiting decisions.

7. Acceptance and traceability

Claude must append the new screen/motion/vision IDs in the accompanying **v2 atlases**, not silently replace the original rows. Coverage table: every item is tagged `DESIGNED`, `PROTOTYPED`, `SOURCE_CONFIRMED`, `PARTNER_DEPENDENT`, `ENGINEERING_READY` or `DEFERRED_WITH_REASON` as appropriate (these states may coexist on distinct axes). For J7 and Scheduling, require their own detailed chapter demos and screen recordings, with actual fixture data and explicit failure states.

The design phase is not complete when all frame names exist. It is complete when a reviewer can navigate the prototyped recruiter, candidate career, scheduling and Q memory flows without a narrator, and the engineering team can identify each component's source, contract, authority and state transitions.